Diagnóstico de Riesgo Coronario mediante Naive Bayes

Rojas Molina Marcell Enrique

2026-10-21

Table of Contents

1. Contexto Clínico y Objetivo 1
1.1 Configuración Inicial y Librerías 1
2. Carga y Exploración Inicial de Datos 2
3. Preprocesamiento y Transformación 3
4. Análisis Exploratorio de Datos (EDA) 4
5. Partición de Datos y Análisis de Valores Atípicos 5
Auditoría de Outliers (Solo en Train) 5
6. Modelado: Naive Bayes 6
7. Evaluación en Test y Optimización del Umbral (Threshold) 7
7.1 Ajuste Clínico del Umbral de Decisión 8

1. Contexto Clínico y Objetivo

El presente análisis utiliza datos clínicos no invasivos para predecir la presencia de enfermedad arterial coronaria (`disease`). El objetivo es construir un modelo de **Naive Bayes** que permita identificar tempranamente a pacientes con alto riesgo, priorizando la **sensibilidad** (minimizar los falsos negativos) para asegurar que los pacientes en riesgo sean derivados a pruebas confirmatorias.

1.1 Configuración Inicial y Librerías

```
# ======================================================================
# 1. Cargar librerías necesarias
# ======================================================================
rm(list = ls())
graphics.off()

# Definición de semilla para reproducibilidad
semilla <- 2026

# Carga de paquetes
```

```r
library(readr)
library(caret)
library(naivebayes)
library(DataExplorer)
library(ggplot2)
library(reshape2)
library(tidyr)
library(dplyr)
library(pROC)    # Para la curva ROC y el AUC en test
```

2. Carga y Exploración Inicial de Datos

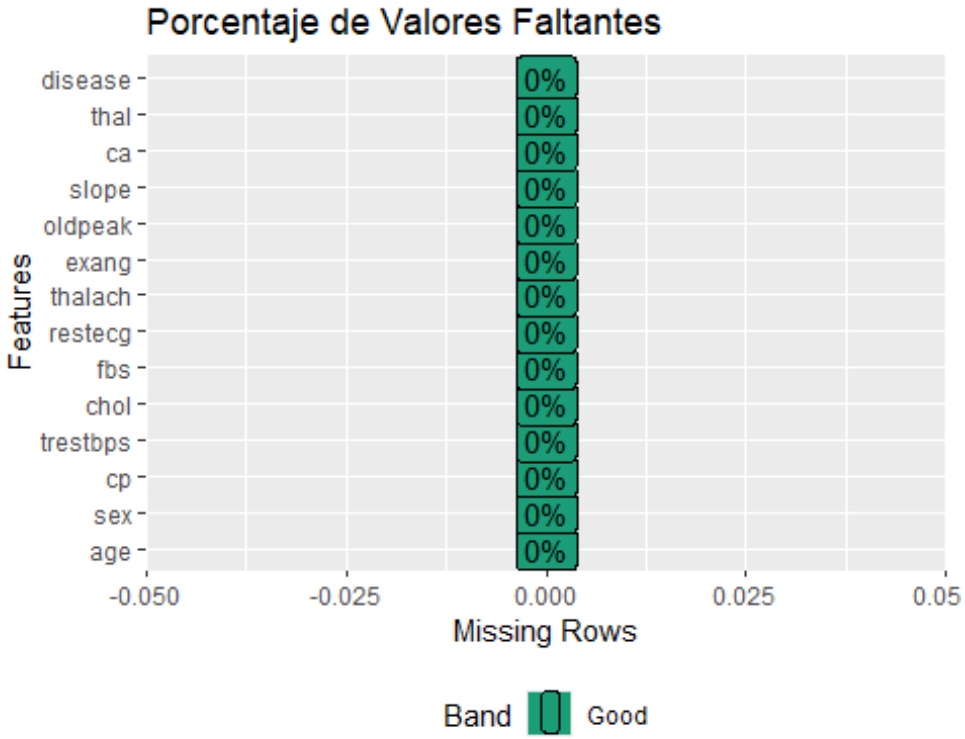
Se procede a cargar el dataset y verificar la integridad de los datos.

```r
# ======================================================================
# 2. Cargar el dataset Heart.csv
# ======================================================================
df <- read.csv("Heart.csv", sep = ";", header = TRUE)

cat("Dimensiones del dataset:", dim(df)[1], "filas y", dim(df)[2],
"columnas.\n\n")
```

```
## Dimensiones del dataset: 297 filas y 14 columnas.
```

```r
# Verificación de valores nulos
plot_missing(df, title = "Porcentaje de Valores Faltantes")   # 0 % de
faltantes en todas las variables -> no se imputa
```



Como se observa, no existen valores nulos en ninguna de las variables, por lo que no es necesario aplicar técnicas de imputación.

3. Preprocesamiento y Transformación

Para que el algoritmo Naive Bayes calcule correctamente las probabilidades a priori y condicionales, es fundamental que las variables categóricas estén tipadas como `factor` y no como texto o números enteros.

```
# Variable objetivo
target <- "disease"
df[[target]] <- as.factor(df[[target]])
levels(df[[target]]) <- c("Sano", "Enfermo")

cat("--- DISTRIBUCIÓN DE LA CLASE OBJETIVO ---\n")

## --- DISTRIBUCIÓN DE LA CLASE OBJETIVO ---

print(prop.table(table(df$disease)) * 100)

##
##     Sano  Enfermo
## 53.87205 46.12795
```

Las clases se encuentran balanceadas (aproximadamente 54% Sanos y 46% Enfermos), por lo que **no se requiere aplicar técnicas de remuestreo sintético** (como SMOTE).

```
# Variables categóricas
cat_cols <- c("sex", "cp", "fbs", "restecg", "exang", "slope", "ca",
"thal")
df[cat_cols] <- lapply(df[cat_cols], as.factor)

# Variables numéricas
num_cols <- c("age", "trestbps", "chol", "thalach", "oldpeak")
df[num_cols] <- lapply(df[num_cols], as.numeric)

# Verificación del supuesto de independencia condicional
datos_num <- Filter(is.numeric, df)
descrCor <- cor(datos_num, use = "complete.obs")
cat("\n--- MATRIZ DE CORRELACIÓN (Variables Numéricas) ---\n")

##
## --- MATRIZ DE CORRELACIÓN (Variables Numéricas) ---

print(round(descrCor, 3))

##             age trestbps  chol thalach oldpeak
## age       1.000    0.290 0.203  -0.395   0.197
## trestbps  0.290    1.000 0.132  -0.049   0.191
## chol      0.203    0.132 1.000   0.000   0.039
```

```
## thalach   -0.395    -0.049 0.000   1.000  -0.348
## oldpeak    0.197     0.191 0.039  -0.348   1.000
```

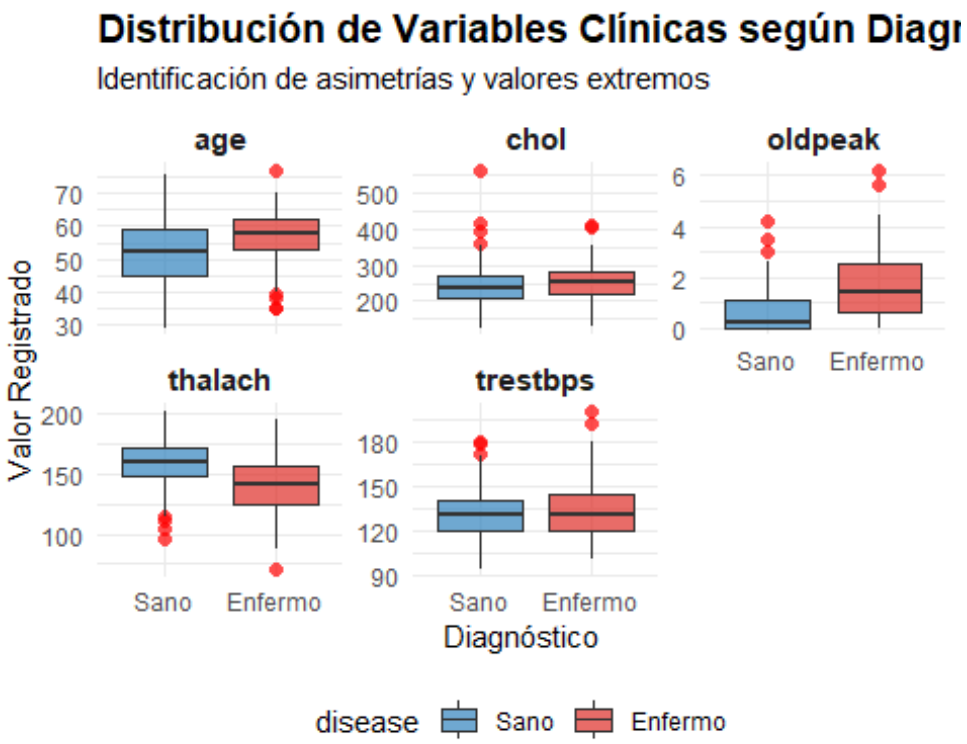
Dado que todas las correlaciones numéricas son $|r| < 0.4$, el supuesto fundamental de independencia condicional de Naive Bayes es razonable para este conjunto de datos.

4. Análisis Exploratorio de Datos (EDA)

Exploramos la distribución de las variables respecto a la condición cardíaca del paciente.

```
# Distribución de numéricas
df_long <- df %>%
  select(all_of(num_cols), disease) %>%
  pivot_longer(cols = -disease, names_to = "Variable", values_to =
"Valor")

ggplot(df_long, aes(x = disease, y = Valor, fill = disease)) +
  geom_boxplot(alpha = 0.7, outlier.colour = "red", outlier.shape = 16,
outlier.size = 2) +
  facet_wrap(~ Variable, scales = "free_y") +
  labs(title = "Distribución de Variables Clínicas según Diagnóstico",
       subtitle = "Identificación de asimetrías y valores extremos",
       x = "Diagnóstico", y = "Valor Registrado") +
  scale_fill_manual(values = c("Sano" = "#3182bd", "Enfermo" =
"#de2d26")) +
  theme_minimal() +
  theme(plot.title = element_text(face = "bold", size = 14),
        strip.text = element_text(face = "bold", size = 11),
        legend.position = "bottom")
```



5. Partición de Datos y Análisis de Valores Atípicos

Realizamos una partición estratificada (70/30) para asegurar que la distribución de la enfermedad se mantenga constante en entrenamiento y prueba.

```
set.seed(semilla)
trainIndex <- createDataPartition(df[[target]], p = 0.7, list = FALSE)
train <- df[trainIndex, ]
test  <- df[-trainIndex, ]

cat("Observaciones en Train:", nrow(train), "| Test:", nrow(test), "\n")
```

```
## Observaciones en Train: 208 | Test: 89
```

Auditoría de Outliers (Solo en Train)

```
cat("--- CONTEO DE OUTLIERS EN ENTRENAMIENTO ---\n")
```

```
## --- CONTEO DE OUTLIERS EN ENTRENAMIENTO ---
```

```
for (v in num_cols) {
  x   <- train[[v]]
  q   <- quantile(x, c(0.25, 0.75))
  iqr <- q[2] - q[1]
  n_iqr <- sum(x < q[1] - 1.5 * iqr | x > q[2] + 1.5 * iqr)
  n_z   <- sum(abs(scale(x)) > 3)
  cat(sprintf("%-9s -> Rango Intercuartílico (IQR): %2d | Z-Score (>3):
```

```
%2d\n", v, n_iqr, n_z))
}
```

```
## age       -> Rango Intercuartílico (IQR):  0 | Z-Score (>3):  0
## trestbps  -> Rango Intercuartílico (IQR):  8 | Z-Score (>3):  2
## chol      -> Rango Intercuartílico (IQR):  4 | Z-Score (>3):  3
## thalach   -> Rango Intercuartílico (IQR):  1 | Z-Score (>3):  1
## oldpeak   -> Rango Intercuartílico (IQR):  2 | Z-Score (>3):  2
```

Decisión Analítica: Se decide **conservar** estos valores atípicos. En el ámbito clínico, valores extremos (ej. colesterol de 564 mg/dl) son fisiológicamente posibles e indicativos de patologías severas. Además, la implementación de Naive Bayes con estimación de densidad por kernel maneja adecuadamente estas colas sin necesidad de truncar información crítica.

6. Modelado: Naive Bayes

En la construcción del modelo, se evita el uso de preprocesamientos de escalado (`center/scale`) y el uso de la notación de fórmula tradicional (`disease ~ .`), ya que esto fuerza la creación de variables *dummy* (One-Hot Encoding). Naive Bayes opera de manera mucho más eficiente y precisa cuando las variables categóricas se mantienen en su formato original de factores.

La optimización de hiperparámetros se dirige a maximizar el **AUC-ROC** en lugar del *Accuracy*.

```
# Validación Cruzada (10-Folds) priorizando probabilidades (ROC)
ctrl <- trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = twoClassSummary
)

predictores <- setdiff(names(train), target)

set.seed(semilla)
modelo_nb <- train(
  x = train[, predictores],
  y = train[[target]],
  method = "naive_bayes",
  trControl = ctrl,
  metric = "ROC",
  tuneGrid = expand.grid(
    laplace   = c(0.5, 1, 2),
    usekernel = c(TRUE, FALSE),
    adjust    = c(0.5, 1, 1.5)
  )
)
```

```r
cat("--- MEJORES HIPERPARÁMETROS ENCONTRADOS ---\n")
```

```
## --- MEJORES HIPERPARÁMETROS ENCONTRADOS ---
```

```r
print(modelo_nb$bestTune)
```

```
##   laplace usekernel adjust
## 7       1     FALSE    0.5
```

7. Evaluación en Test y Optimización del Umbral (Threshold)

El modelo final se evalúa sobre el conjunto de prueba (30% reservado).

```r
prob_test <- predict(modelo_nb, newdata = test[, predictores], type =
"prob")
pred_test <- predict(modelo_nb, newdata = test[, predictores])

# Matriz de Confusión Estándar (Umbral 0.5)
cm_test <- confusionMatrix(pred_test, test[[target]], positive =
"Enfermo")
cat("Sensibilidad Inicial (Umbral 0.5):",
round(cm_test$byClass["Sensitivity"], 3), "\n")
```

```
## Sensibilidad Inicial (Umbral 0.5): 0.756
```

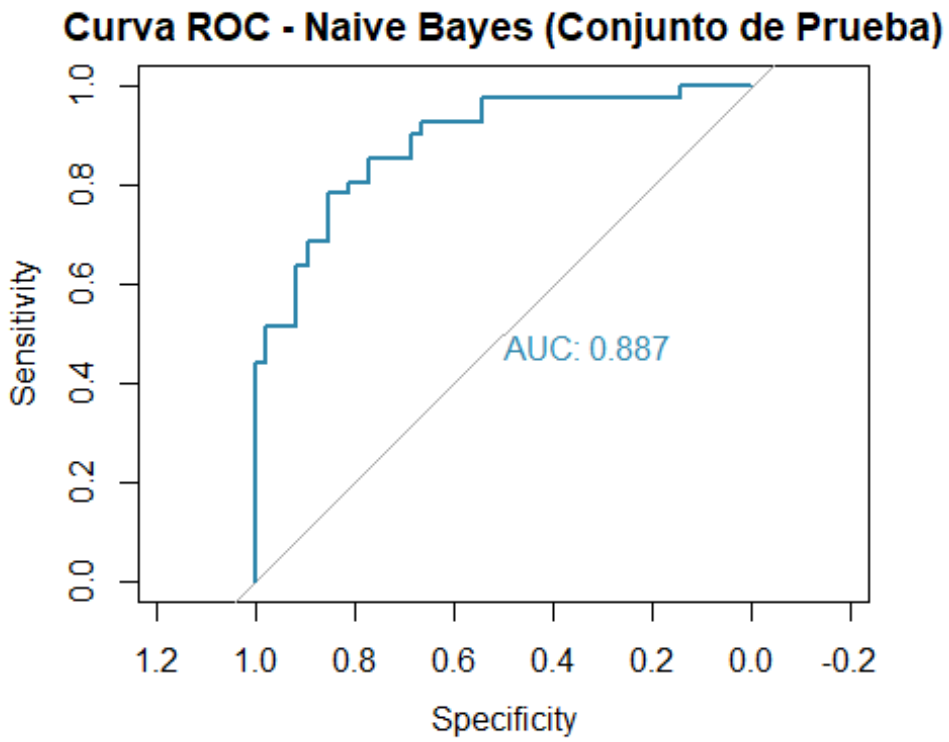
```r
cat("Especificidad Inicial (Umbral 0.5):",
round(cm_test$byClass["Specificity"], 3), "\n")
```

```
## Especificidad Inicial (Umbral 0.5): 0.854
```

```r
# Análisis de la Curva ROC
roc_test  <- roc(test[[target]], prob_test$Enfermo, levels = c("Sano",
"Enfermo"))
cat("Área bajo la curva (AUC-ROC) en Test:", round(auc(roc_test), 4),
"\n\n")
```

```
## Área bajo la curva (AUC-ROC) en Test: 0.8867
```

```r
plot(roc_test, main = "Curva ROC - Naive Bayes (Conjunto de Prueba)",
     col = "#2E86AB", lwd = 2, print.auc = TRUE)
```



7.1 Ajuste Clínico del Umbral de Decisión

En diagnóstico médico, un **falso negativo** (paciente enfermo clasificado como sano) tiene un costo severo para la salud del paciente. Por ello, se evalúa reducir el umbral de decisión habitual (0.5) para incrementar la **Sensibilidad** del modelo, aceptando una ligera penalización en la especificidad (falsos positivos derivables a exámenes de rutina).

```
cat("--- RENDIMIENTO SEGÚN UMBRAL DE CLASIFICACIÓN ---\n")
```

```
## --- RENDIMIENTO SEGÚN UMBRAL DE CLASIFICACIÓN ---
```

```
for (u in c(0.3, 0.4, 0.5)) {
  pred_u <- factor(ifelse(prob_test$Enfermo >= u, "Enfermo", "Sano"),
                   levels = c("Sano", "Enfermo"))
  cm_u <- confusionMatrix(pred_u, test[[target]], positive = "Enfermo")

  cat(sprintf("Umbral %.1f | Sensibilidad (Recall): %.3f | Especificidad:
%.3f | Accuracy: %.3f\n",
              u, cm_u$byClass["Sensitivity"],
cm_u$byClass["Specificity"],
              cm_u$overall["Accuracy"]))
}
```

```
## Umbral 0.3 | Sensibilidad (Recall): 0.805 | Especificidad: 0.771 |
Accuracy: 0.787
## Umbral 0.4 | Sensibilidad (Recall): 0.805 | Especificidad: 0.792 |
```

```
Accuracy: 0.798
## Umbral 0.5 | Sensibilidad (Recall): 0.756 | Especificidad: 0.854 |
Accuracy: 0.809
```

Conclusión: Bajar el umbral de probabilidad a **0.4** mejora significativamente la Sensibilidad del modelo, permitiendo capturar a una mayor cantidad de pacientes enfermos, siendo esta la configuración recomendada para un entorno de cribado clínico (screening).